L24 — Evaluation of Postfix Expressions

Unit: 2 | **Chapter:** Stacks & Recursion | **BT Level:** BT5 | **CO:** CO2, CO3

Learning Objectives

- Distinguish between infix, prefix, and postfix notation
- Evaluate a postfix expression using a stack
- Trace through postfix evaluation step by step
- Handle multi-digit operands and extended operators

---

1. Expression Notations

An arithmetic expression like `3 + 4 * 2` can be written in three notations:

Notation	Position of Operator	Example
Infix	Between operands	`3 + 4 * 2`
Prefix (Polish)	Before operands	`+ 3 * 4 2`
Postfix (Reverse Polish)	After operands	`3 4 2 * +`

Why Postfix?

- **No parentheses needed** — precedence is implicit in the notation
- **No ambiguity** — fully unambiguous
- **Easier for computers** to evaluate using a stack
- Used in: calculators (HP-style), compilers, virtual machine bytecode

Infix vs Postfix equivalents:

Infix	Postfix
`A + B`	`A B +`
`A + B * C`	`A B C * +`
`(A + B) * C`	`A B + C *`
`(A + B) * (C - D)`	`A B + C D - *`

---

2. Algorithm: Postfix Evaluation

```
Algorithm evaluate_postfix(expression):
  stack = empty stack

  for each token in expression:
```

```
    if token is an operand (number):
        push(token)
    elif token is an operator (+, -, *, /):
        operand2 = pop()   # Right operand (popped first)
        operand1 = pop()   # Left operand
        result = apply operator to (operand1, operand2)
        push(result)

return pop()   # Final result
```

Critical: The first popped value is the **right operand**, the second is the **left operand**.

---

3. Implementation

```
def evaluate_postfix(expression):
    """
    Evaluate a postfix expression.
    Tokens are separated by spaces.
    """
    stack = []
    tokens = expression.split()

    operators = {
        '+': lambda a, b: a + b,
        '-': lambda a, b: a - b,
        '*': lambda a, b: a * b,
        '/': lambda a, b: int(a / b),  # Integer division
        '^': lambda a, b: a ** b,
        '%': lambda a, b: a % b,
    }

    for token in tokens:
        if token in operators:
            if len(stack) < 2:
                raise ValueError("Invalid postfix expression")
            b = stack.pop()    # Right operand
            a = stack.pop()    # Left operand
            result = operators[token](a, b)
            stack.append(result)
        else:
            try:
                stack.append(int(token))
            except ValueError:
                raise ValueError(f"Invalid token: {token}")

    if len(stack) != 1:
        raise ValueError("Invalid postfix expression")

    return stack.pop()
```

---

4. Detailed Traces

Example 1: `5 3 + 2 *`

*(Equivalent infix: (5 + 3) * 2)*

Token	Action	Stack
5	Push 5	[5]
3	Push 3	[5, 3]
+	Pop 3, Pop 5 → 5+3=8 → Push 8	[8]
2	Push 2	[8, 2]
*	Pop 2, Pop 8 → 8*2=16 → Push 16	[16]

Result: 16 ✓

Example 2: `2 3 4 * + 5 –`

*(Equivalent infix: 2 + 3 * 4 - 5)*

Token	Action	Stack
2	Push 2	[2]
3	Push 3	[2, 3]
4	Push 4	[2, 3, 4]
*	Pop 4, Pop 3 → 3*4=12 → Push 12	[2, 12]
+	Pop 12, Pop 2 → 2+12=14 → Push 14	[14]
5	Push 5	[14, 5]
–	Pop 5, Pop 14 → 14-5=9 → Push 9	[9]

Result: 9 ✓
Verify: 2 + 3*4 - 5 = 2 + 12 - 5 = 9 ✓

Example 3: `8 2 / 3 –`

(Equivalent infix: 8 / 2 - 3)

Token	Action	Stack
8	Push 8	[8]
2	Push 2	[8, 2]
/	Pop 2, Pop 8 → 8/2=4 → Push 4	[4]
3	Push 3	[4, 3]
–	Pop 3, Pop 4 → 4-3=1 → Push 1	[1]

Result: 1 ✓

---

5. Handling Operator Associativity

For left-to-right operators: pop right operand first, then left.
For the – and / operators, order matters:

```
Expression: 6 3 -
  b = pop() = 3 (right)
  a = pop() = 6 (left)
  result = a - b = 6 - 3 = 3  ✓ (not 3 - 6 = -3)
```

---

6. Complete Test Suite

```
test_cases = [
    ("5 3 +", 8),
    ("5 3 -", 2),
    ("5 3 *", 15),
    ("8 2 /", 4),
    ("2 3 4 * +", 14),
    ("5 1 2 + 4 * + 3 -", 14),  # 5 + (1+2)*4 - 3
    ("2 3 ^ 1 +", 9),           # 2^3 + 1
]

for expr, expected in test_cases:
    result = evaluate_postfix(expr)
    status = "✓" if result == expected else "✗"
    print(f"{status} evaluate_postfix('{expr}') = {result} (expected {e
```

---

7. Complexity Analysis

Aspect	Complexity
Time	O(n) — each token processed once
Space	O(n) — stack holds at most n/2 operands

For an expression with n tokens: n/2 operands + n/2 operators → maximum stack depth = n/2 → O(n) space.

Summary

- Postfix notation places operators after operands — no parentheses needed, no ambiguity.
- **Evaluation algorithm:** scan left to right; push operands, pop and apply on operators.
- **Key:** when applying operator, first pop = right operand, second pop = left operand.
- Time: O(n), Space: O(n) — each token processed at most once.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.1 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.4 (Springer, 2020)
- LeetCode #150 — Evaluate Reverse Polish Notation